

ASSIGNMENT #01 ANALYSIS OF ALGORITHM TIME COMPLEXITY



Mo Tu We Th Fr Sa Su

Memo No. _____

Date 19 / 10 / 25

Q#01: Calculate Time Complexity (Best n Worst Case both)

```

Void B_S (int arr [], int n) {
1   for (int i=0; i<n-1; i++) {
2       for (int j=0; j<n-i-1; j++) {
3           if (arr[j] > arr[j+1]) {
4               int temp = arr[j];
5               arr[j] = arr[j+1];
6               arr[j+1] = temp;
            }
        }
    }
}
    
```

kill n (+ false case)
 $\frac{n(n-1)}{2}$ (Bubble Sort)

Bubble Sort Logic:

eg; iteration=1

let arr[5 2 8 1] n=4

1 5 2 8 1

2 2 5 8 1

3 2 5 8 1

2 5 1 8

n=4 , n-1=3 (Comparisons)

Iterations	Comparison
1 st	(n-1)
2 nd	(n-2)
3 rd	(n-3)
...	...
Last	1

for
Array → n elements
 $= (n-1) + (n-2) + (n-3) + \dots + 1$
 $= \frac{n(n-1)}{2}$
 $= n^2$

Loops = 2 = $n * n = n^2$



Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

0	1	2	3
5	2	8	1

where $j = n - 1$

→ pass 1 ($i = 0$)

$j = 0$

compare $arr[0] > arr[1]$

$5 > 2$
swap [2 5 8 1]

$j = 1$

compare $arr[1] > arr[2]$

• false ($no\ swap$) $5 > 8$

$j = 2$

compare $arr[2] > arr[3]$

$8 > 1$
swap [2 5 1 8]

Ops	Best Case	Worst Case
c_1	n	n
c_2	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
c_3	$\frac{n(n-1)}{2}$	//
c_4	0	//
c_5	0	//
c_6	0	//

→ pass 2 ($i = 1$)

$j = 0$

compare $arr[0] > arr[1]$

• false ($no\ swap$) $2 > 5$

$j = 1$

compare $arr[1] > arr[2]$

$5 > 1$
swap [2 1 5 8]

Time Complexity:

Best case : $O(n^2)$

Worst case : $O(n^2)$

→ pass 3 ($i = 2$)

$j = 0$

compare $arr[0] > arr[1]$

$2 > 1$

Swap [1 2 5 8]

(modified Binary Search)

- First Occurance - of key

- Second Occurance - First index



Mo Tu We Th Fr Sa Su

Memo No. _____

Date

/

/

+ key

Q# 02 : Solve problem in $O(\log n)$

Algorithm (pseudo code)

function findFirst(arr, key, low, high):

result = -1

while low $\leq$ high

mid = low + (high - low) / 2

if arr[mid] == key:

result = mid

high = mid - 1 // go left to find 1st Occurance

else if arr[mid] < key:

low = mid + 1

else:

high = mid - 1

return result

function findSecondOccurance(arr, n, key):

first = findFirst(arr, key, 0, n - 1)

if first == -1

cout << "key not found"

else:

second = findFirst(arr, key, first + 1, n - 1)

if second == -1

cout << "2nd Occurance doesn't exist"

else:

cout << "2nd Occurance at index", second



Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

Time Complexity:

Two binary searches $\rightarrow O(\log n) + O(\log n) = O(\log n)$

let $arr = [2 \ 4 \ 4 \ 4 \ 6 \ 7 \ 8]$ where $key = 4$

① Find First Occurrence of 4

- $low = 0$, $high = 6$
 $mid = 3 \rightarrow arr[3] = 4 \rightarrow \text{found}$, set $result = 3$, $high = 2$
- $low = 0$, $high = 2$
 $mid = 1 \rightarrow arr[1] = 4 \rightarrow \text{found}$, $result = 1$, $high = 0$
- $low = 0$, $high = 0$
 $mid = 0 \rightarrow arr[0] = 2 < 4 \rightarrow low = 1$
- $low > high \rightarrow \text{stop}$

First Occurrence Index $x = 1$

② Find First Occurrence from Index 2

- $low = first + 1 = 2$, $high = 6$
 $mid = 4 \rightarrow arr[4] = 6 > 4 \rightarrow high = 3$
- $low = 2$, $high = 3$
 $mid = 2 \rightarrow arr[2] = 4 \rightarrow \text{found}$, $result = 2$, $high = 1$
- $low > high \rightarrow \text{stop}$

Second Occurrence Index $x = 2$

Output = Second Occurrence at Index $x = 2$



Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

Key Appears once - example:

let arr [1 3 5 7] where
key = 3

First Occurrence = 1

Search from index 2 to 3 for 3 $\Rightarrow$ Not Found

Output: second Occurrence doesn't exist

key not present - example:

let arr [1 3 5 7] where
key = 4

First Occurrence = -1

Output: key not Found